

RZ/A2M Group

I2C Bus Interface EEPROM Read/Write Example

Introduction

This document describes an example of using communication (single-master transmission/reception) via the I2C bus

interface (hereinafter called "IIC") of the RZ/A2M to implement read and write access to the EEPROM.

It provides a comprehensive overview of the software. For further details please refer to the software itself.

The user is assumed to be equipped with an RZ/A2M CPU board.

Target Device

RZ/A2M

When using this application note with other Renesas MCUs, careful evaluation is recommended after making modifications to comply with the alternative MCU.

List of Abbreviations and Acronyms

Abbreviation	Full Form
API	Application Programming Interface
ARM	Advanced RISC Machines
BSS	Block Started by Symbol
BTAC	Branch Target Address Cache
CPG	Clock Pulse Generator
CPU	Central Processing Unit
DACR	Domain Access Control Register
DMAC	Direct Memory Access Controller
FIFO	First In First Out
FIQ	Fast Interrupt
FPU	Floating Point Unit
GPIO	General Purpose Input Output
INTC	Interrupt Controller
I/O	Input/Output
IRQ	Interrupt Request
L1	Level 1 (primary cache)
L2	Level 2 (secondary cache)
LED	Light Emitting Diode
LSB	Least Significant Bit
MCU	Microcontroller Unit
MMU	Memory Management Unit
NMI	Non-Maskable Interrupt
OS	Operating System
OSTM	Operating System Timer Module
PC	Personal Computer
PLL	Phase-Locked Loop
RAM	Random-Access Memory
ROM	Read Only Memory
RTC	Real Time Clock
SC	Smart Configurator
SCIFA	Serial Communications Interface with FIFO
SCTLR	System Control Register
SDRAM	Synchronous Dynamic Random-Access Memory
SPI	Serial Peripheral Interface
SPIBSC	SPI Multi I/O Bus Controller
STB	Standby module
STDIO	Standard Input/Output

TLB	Translation Lookaside Buffer
TTBCR	Translation Table Base Control Register
TTBR	Translation Table Base Register
USB	Universal Serial Bus
VBAR	Vector Base Address Register
XTAL	Crystal

Table 1-1 List of Abbreviations and Acronyms

Contents

1. Specifications	6
2. Proven Operating Conditions	8
3. Reference Application Notes	9
4. Hardware	10
4.1 Hardware Configuration	10
4.2 EEPROM Specifications	10
4.3 Pins Used	11
5. Software	12
5.1 Operation Overview	12
5.1.1 Writing to EEPROM	12
5.1.2 Reading from EEPROM	13
5.1.3 Acknowledge Polling	13
5.2 Peripheral Function Setting and Memory Allocation in Sample Program	14
5.2.1 Peripheral Function Settings	14
5.2.2 Section Assignment of the sample program	16
5.3 Interrupts	18
5.4 Fixed-width types	18
5.5 Constants	19
5.6 Variables	22
5.7 Functions	23
5.8 Function Specification	24
5.8.1 EEPROM Control Function	24
5.8.2 Console Function	25
5.9 Flowcharts	28
5.9.1 sample_riic_eeprom_init	28
5.9.2 Sample_RIIC_Eeprom_UnInit	28
5.9.3 Sample_RIIC_Eeprom_Write	29
5.9.4 Sample_RIIC_Eeprom_Read	30
5.9.5 r_riic_ackpoll	31
5.9.6 cmd_version	32
5.9.7 cmd_write_eeprom	33
5.9.8 cmd_read_eeprom	34
5.9.9 cmd_help	35
5.9.10 cmd_exit	35
6. Sample Code	36
7. Reference Documents	36

Revision History37

1. Specifications

After a device reset, the following devices are initialized by the program located in the serial flash memory assigned in SPI Multi I/O bus space.

- SPI Multi I/O Bus Controller
- Clock Pulse Generator
- Interrupt Controller
- Bus State Controller
- OS Timer
- General I/O Ports
- Memory Management Unit
- Primary Cache
- Secondary Cache

In this application note, the SPI Multi I/O Bus Controller, Clock Pulse Generator, Interrupt Controller, OS Timer, Serial Communication Interface with FIFO, General I/O Ports, Power-down Mode, I2C Bus Interface and Memory Management Unit are referred to as SPIBSC, CPG, INTC, OSTM, SCIFA, RIIC, STB and MMU, respectively.

Table 1.1 lists the peripheral functions supported in this application note. Figure 1.1 shows the operation overview.

Table 1.1 Peripheral devices used

Peripheral device	Usage
SPI Multi I/O Bus Controller (SPIBSC)	Configure SPIBSC as external address space read mode and generate the signal for CPU to read data from serial flash memory assigned in SPI Multi I/O bus space directly
Clock Pulse Generator (CPG)	Generate the operating frequency of RZ/A2M
Interrupt Controller (INTC)	Control the interrupts used by OSTM channel 0 and SCIFA channel 4
OS Timer (OSTM)	Manage the timer for blinking the LED using OSTM channel 0
Serial Communication Interface with FIFO (SCIFA)	Control the communications between the RZ/A2M and a host PC using SCIFA channel 4
I2C Bus Interface (RIIC)	Control the I2C communications between the RZ/A2M and EEPROM
General I/O Ports (GPIO)	Assign the multiplexed pin functions for SCIFA channel 4 and RIIC channel 3 Control I/O pins for blinking the LED
Power-down Mode (STB)	Disable the RZ/A2M peripheral I/O module standby Enable writing to the on-chip data retention RAM
Memory Management Unit (MMU), Primary (L1) Cache and Secondary (L2) Cache	Generate MMU translation table for configuring the area where L1 cache is enabled, memory attribute of each region of memory (i.e. Normal, Device, or Strongly-Ordered) Enable L1 and L2 Cache

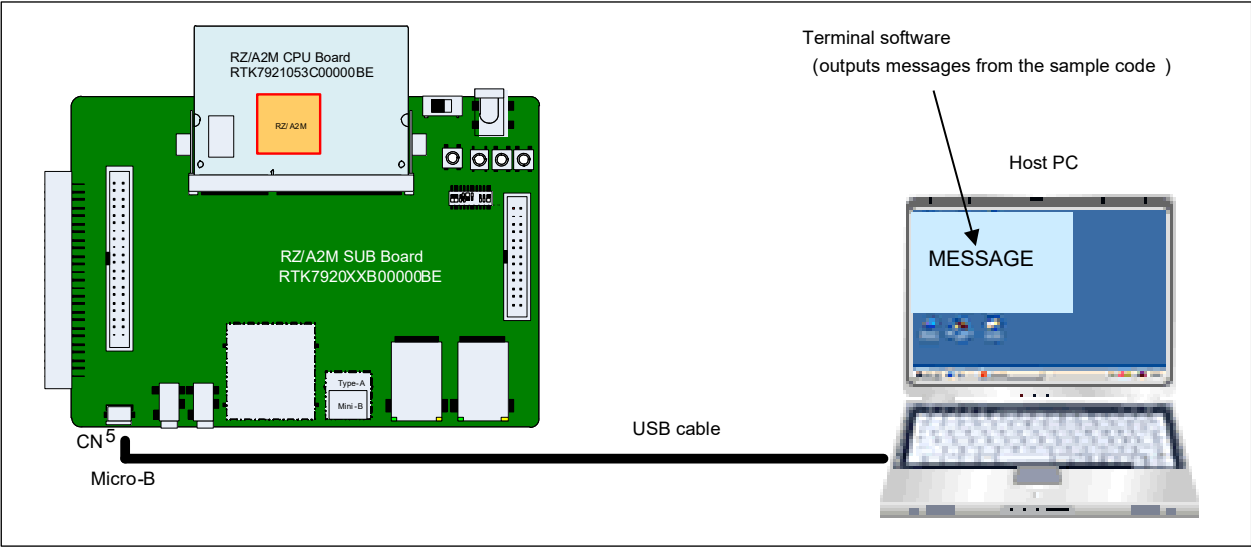


Figure 1.1 Operation check conditions

2. Proven Operating Conditions

The sample program accompanying this application note has been verified and works as expected under the conditions shown in Table 2.1.

Table 2.1 Proven Operating Conditions (1/2)

Item	Description
MCU used	RZ/A2M
Operating frequency (Note)	CPU Clock ($I\phi$) : 528MHz Image processing clock ($G\phi$) : 264MHz Internal Bus Clock ($B\phi$) : 132MHz Peripheral Clock 1 ($P1\phi$) : 66MHz Peripheral Clock 0 ($P0\phi$) : 33MHz QSPI0_SPCLK : 66MHz CKIO : 132MHz
Operating voltage	Power supply voltage (I/O): 3.3 V Power supply voltage (either 1.8V or 3.3V I/O (PVcc_SPI)) : 3.3V Power supply voltage (internal): 1.2 V
Integrated development environment	e2 studio 2025-12
C compiler	“ Arm GNU Tool chain 12.2.Rel1” compiler options(except directory path) Release: -mcpu=cortex-a9 -march=armv7-a -marm -mlittle-endian -mfloat-abi=hard -mfpu=neon -mno-unaligned-access -Os -ffunction-sections -fdata-sections -Wnull-dereference -Wstack-usage=100 -fabi-version=0 Hardware Debug: -mcpu=cortex-a9 -march=armv7-a -marm -mlittle-endian -mfloat-abi=hard -mfpu=neon -mno-unaligned-access -Og -ffunction-sections -fdata-sections -Wnull-dereference -g3 -Wstack-usage=100 -fabi-version=0

Note: The operating frequency used in clock mode 1 (Clock input of 24MHz from EXTAL pin)

Table 2.2 Proven Operating Conditions (2/2)

Item	Description
Operation mode	Boot mode 3 (Serial Flash boot 3.3V)
Terminal software communication settings	• Communication speed: 115200bps • Data length: 8 bits • Parity: None • Stop bits: 1 bit • Flow control: None
Board to be used	RZ/A2M CPU board RTK7921053C00000BE RZ/A2M SUB board RTK79210XXB00000BE
Device (functionality to be used on the board)	• Serial flash memory allocated to SPI multi-I/O bus space - Manufacturer : Macronix Inc. - Model Name : MX25L51245GXD • RL78/G1C (USB-to-serial converter, which is used for communicating with host PC) • LED1 • EEPROM - Manufacturer : Renesas electronics. - Model Name : R1EX24128ASAS0A

3. Reference Application Notes

In this chapter, application note referenced in this note are listed:

- RZ/A2M Group Example of booting from serial flash memory (R01AN4333JJ)

4. Hardware

4.1 Hardware Configuration

Figure 4.1 shows a Connection Example in which the RZ/A2M is used as the master device and the EEPROM (R1EX24128ASAS0A) as the slave device. In the connection example, the pins are processed such that the device address code of the EEPROM is "B'000".

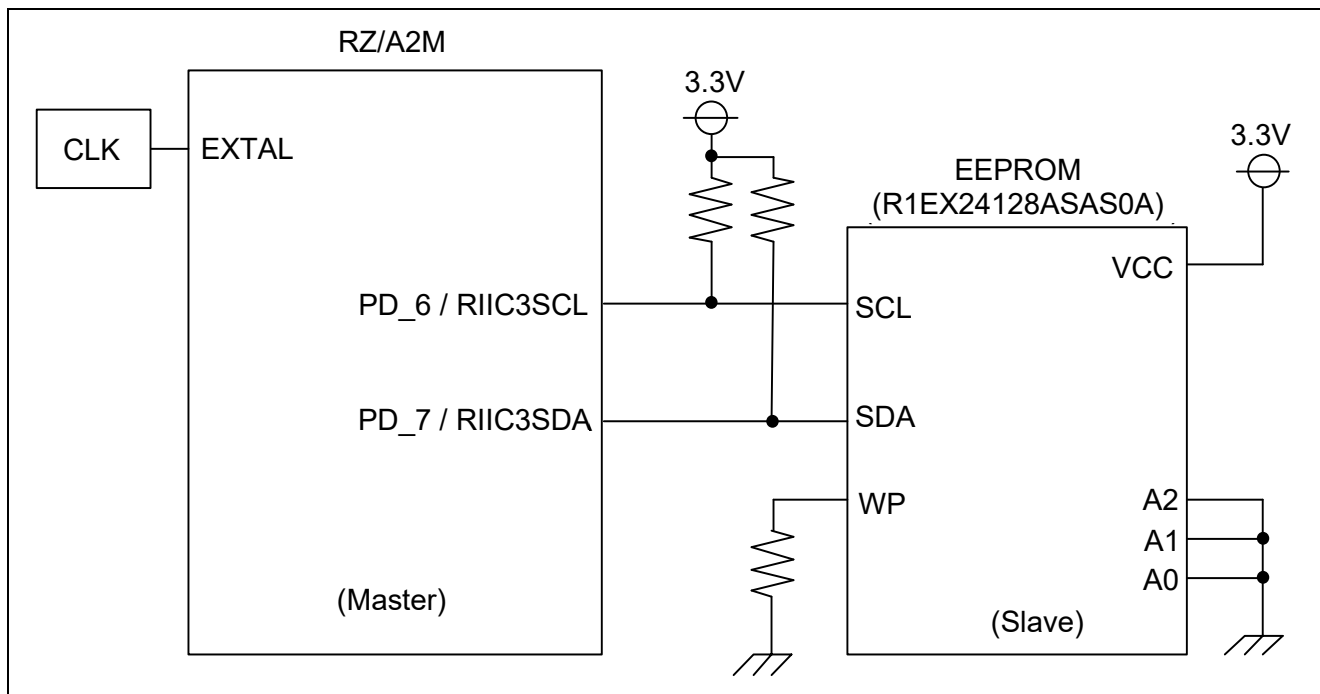


Figure 4.1 Connection Example

4.2 EEPROM Specifications

Table 4.1 lists the EEPROM Specifications.

Table 4.1 EEPROM Specifications

Item	Contents
Product No.	R1EX24128ASAS0A
Capacity	16KB (16K words × 8 bits)
Slave address	H'A0 Device code (bit7 to bit4): B'1010 Device address code (bit3 to bit1): B'000 (Set by pins A2 to A0. Connected to GND on board.) R/W# bit (bit0): Write when "0", read when "1"
Write operation	Page write function support (Support automatic incrementing for addresses (A5 to A0) within pages of up to 64bytes)
Read operation	Sequential read function support (Enable continuous data output with incrementing the address when ACK is input after output of each data block)

4.3 Pins Used

Table 4.2 Lists the Pins Used and Their Functions.

Table 4.2 Pins Used and Their Functions

Pin Name	I/O	Description
MD_BOOT2	Input	Boot mode selection (Select boot mode 3) MD_BOOT2: "L", MD_BOOT1: "H", MD_BOOT0: "H"
MD_BOOT1	Input	
MD_BOOT0	Input	
QSPI0_SCLK	Output	Clock output to serial flash memory
QSPI0_SSL	Output	Slave selection for serial flash memory
QSPI0_IO0	Input/Output	Data 0 pin for serial flash memory
QSPI0_IO1	Input/Output	Data 1 pin for serial flash memory
QSPI0_IO2	Input/Output	Data 2 pin for serial flash memory
QSPI0_IO3	Input/Output	Data 3 pin for serial flash memory
RPC_RESET# (Note)	Output	Reset control signal output to serial flash memory
P6_0	Output	LED blinking
RxD4 (P9_1)	Input	Serial receive data signal
TxD4 (P9_0)	Output	Serial transmit data signal
RIIC3SCL (PD_6)	Input/Output	RIIC serial clock input/output to the EEPROM
RIIC3SDA (PD_7)	Input/Output	RIIC data clock input/output to the EEPROM

Note: The "#" symbol denotes negative logic (active low)

5. Software

5.1 Operation Overview

The software provides initial setting to the peripheral function modules (RIIC, SCIFA, GPIO, STB, and INTC) to enable sending and receiving of data on RIIC 3 at bit rate of 400kbps, and the master transmit operation on RIIC channel 3 is used to write 1 - 256bytes of data to decided address on EEPROM. Also, 1 - 256bytes of data is read from the decided address on the EEPROM.

The page write function is used when writing to the EEPROM. The sequential read function is used when reading data from the EEPROM

When transmitting the slave address, "B'1010" is used as the device code and "B'000" as the device address code of the EEPROM.

To the driver, please set "B'10100000" which shifted the above slave address 1 bit to the left as a slave address. The R/W# bit of slave address is given by the driver, so this bit need not be set this bit in the application.

5.1.1 Writing to EEPROM

The master transmission mode is used to perform 64 bytes page write to the EEPROM.

A start condition is issued on RIIC channel 3, and next the slave address (device code and device address code) is transmitted. Following the slave address, "0" is transmitted as the R/W# bit, indicating a write operation. Then, a 2byte memory address (A13 to A0) set by application and the data up to 64 bytes address boundary are transmitted on after another. After writing to the 64 bytes boundary, issue a stop condition and end the transmission once. Thereafter, the start condition is issued again, and the next 64 bytes data is transmitted. This process is repeated until data is written up to the decided data size by application.

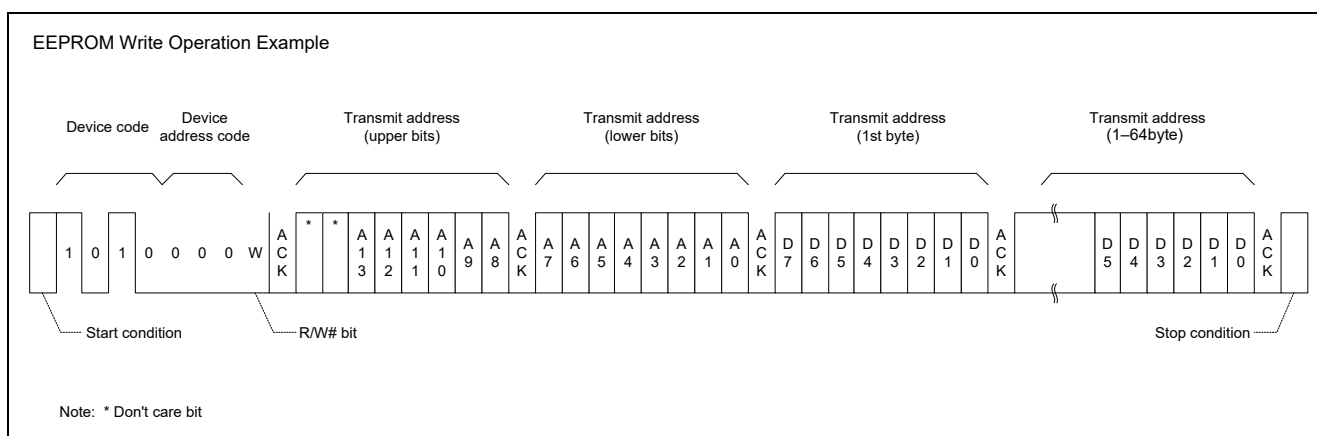


Figure 5.1 EEPROM Write Operation Example (Page Write)

5.1.2 Reading from EEPROM

The master transmission mode and the master receive mode are used to perform a sequential read up to 256 bytes.

Master transmit is used to issue a start condition on RIIC channel 3, and next the slave address (device code and device address code) is transmitted. Following the slave address, "0" is transmitted as the R/W# bit, indicating a write operation. Then, a 2-byte memory address is transmitted. The 2-byte memory address transmitted at this time specifies the address in the EEPROM from which data is to be read.

Next, the restart condition is issued, and the slave address is transmitted once again. Following the slave address, "1" is transmitted as the R/W# bit, indicating a read operation, and a transition to master receive mode occurs. After transmission of the R/W# bit, the clock is output on the I2C bus, and read data from the EEPROM is received in succession. The RIIC transmits an ACK for each byte of received data as the read data is being received, with the exception of the final byte of data, after which a NACK is transmitted. After transmission of the NACK, the stop condition is issued, and the bus is released. In the sample code, the memory address to which data is read is set by the application.

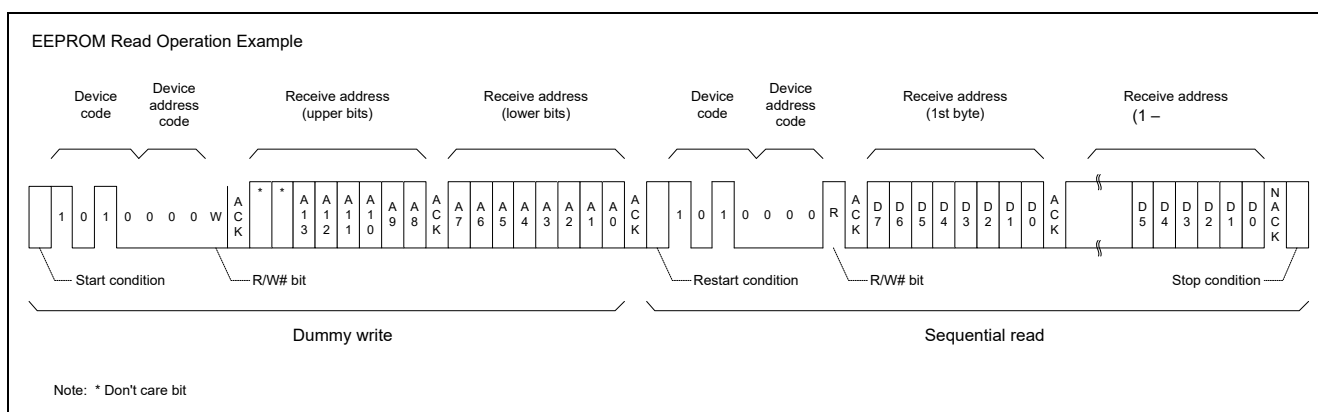


Figure 5.2 EEPROM Read Operation Example (Sequential Read)

5.1.3 Acknowledge Polling

After writing to the EEPROM and, the sample code performs acknowledge polling to wait for memory write completion to the inside of the EEPROM. In acknowledge polling, a dummy read from the address 0x0000 in the EEPROM is performed. If the read address is transmitted when it is still writing to the EEPROM, the return value of the RIIC driver function (control()) is DRV_ERROR since the EEPROM transmits NACK. If writing to the EEPROM has ended at the time of transmitting the read address, the return value of the RIIC driver function (control()) is DRV_SUCCESS since the EEPROM transmits ACK. Therefore, acknowledge polling is executed by performing a dummy read until the return value becomes DRV_SUCCESS.

5.2 Peripheral Function Setting and Memory Allocation in Sample Program

5.2.1 Peripheral Function Settings

Table 5.1 lists the peripheral Function Settings.

Peripheral	Setting
CPG	CPU clock: 1/2 of the PLL circuit frequency Internal bus clock: 1/8 of the PLL circuit frequency Peripheral clock 1: 1/16 of the PLL circuit frequency Each clock takes the following value under Clock Mode 1 (the frequency division ratio by divider 1 is $\frac{1}{2}$, and the input clock frequency is multiplied by 88) when the input clock frequency is 24MHz: • CPU clock ($I\phi$): 528MHz • Image processing clock ($G\phi$): 264MHz • Internal bus clock ($B\phi$): 132MHz • Peripheral clock 1 ($P1\phi$): 66MHz • Peripheral clock 0 ($P0\phi$): 33MHz • QSPI0 SPCLK: 66MHz (when selecting $B\phi$ as output clock) • CKIO clock: 132MHz (when selecting $B\phi$ as output clock)
STB	Make on-chip Data Retention RAM writable and enable clocks to the peripheral modules
GPIO	Configure the multiplexed pin function of PORT6 and PORT9 as follows: • P6_0: LED blinking • P9_1: RxD4, P9_0: TxD4 • PD_6: RIIC3SCL, PD_7: RIIC3SDA
OSTM	Configure the channel 0 as interval timer mode: Counter is configured so that OSTM fires an interrupt every 1ms (when $P1\phi$ is specified as 66MHz)
INTC	Initial setup of INTC Interrupt handler registration and invocation: • Registration of OSTM channel 0 interrupt handler (Interrupt ID: 88) • Registration of SCIFA channel 4 interrupt handler and its invocation (Interrupt ID: 322, 323) • Registration of RIIC channel 3 interrupt handler and its invocation (Interrupt ID: 288, 289, 290, 291, 292, 293, 294, 295)
SCIFA	Configure SCIFA channel 4 as asynchronous communication mode Serial communication setting: • Data length: 8-bits • Stop bits: 1 bit • Parity: None • LSB-first transfer SCIFA is configured as follows (when $P1\phi$ is 66MHz): • Clock source: No frequency division • Baud rate generator: double-speed mode • Operating on the base clock with 8 times the bit rate • Configures the appropriate bit rate for a baud rate of 115200bps Thus, the bit rate should become -0.53%
RIIC	Channel 3 Master/slave: Master mode Address format: 7-bit address format SCL frequency: 400KHz

(IIC ϕ set to P1 ϕ /4 when P1 ϕ = 66.67 MHz, SCL clock high width to 19 clock cycles, SCL clock low width to 19 clock cycles, resulting in a SCL frequency of 400KHz*)
 Internal reference clock (IIC ϕ): P1 ϕ /4
 SCL synchronous circuit enable: Enabled
 Digital noise filter: Disabled
 Arbitration-lost detection: Disabled
 Timeout detection: Disabled
 Transmit complete interrupt (INTRIICTEI3): Enabled, priority = 9
 Receive-data-full interrupt (INTRIICRI3): Enabled, priority = 9
 Transmit-data-empty interrupt (INTRIICTI3): Enabled, priority = 9
 Detection of a stop condition interrupt (INTRIICSPI3): Enabled, priority = 9
 Detection of a start condition interrupt (INTRIICSTI3): Enabled, priority = 9
 Reception of a NACK interrupt (INTRIICNAKI3): Enabled, priority = 9
 Arbitration interrupt (INTRIICALI3): Enabled, priority = 9
 Timeout interrupt (INTRIICTMOI3): Enabled, priority = 9

Table 5.1 Peripheral Function Settings

5.2.2 Section Assignment of the sample program

Table 5.2 lists the sections used in the sample code. Figure 5.4 shows the section assignment for the initial condition of the sample code and the condition after using INITSCCT function.

Table 5.2 Memory Area Used

Output section Name	Input section Name Input Object Name	Description	Loading Area	Execution Area
LOAD_MODULE1	VECTOR_TABLE	Exception processing vector table	FLASH	FLASH
LOAD_MODULE2	* /r_cpg/* .o (.text .rodata .data)	Program code area for CPG	FLASH	LRAM
	* /rza_io_regrw.o (.text .rodata .data)	Program code area for function of I/O register access		
LOAD_MODULE3	* /r_cpg/* .o (.bss)	Data area without initial value for CPG	-	LRAM
	* /rza_io_regrw.o (.bss)	Data area without initial value for function of I/O register access		
LOAD_MODULE4	RESET_HANDLER	Program code area of reset handler processing	FLASH	FLASH
	INIT_SECTION */sections.o	Program code area of section initialization processing		
.data	VECTOR_MIRROR_TABLE	Exception processing vector table	FLASH	LRAM
	* /r_intc* .o (.text .rodata .data)	Program code area for INTC Driver		
	IRQ_FIQ_HANDLER	IRQ/FIQ handler processing		
.bss	none	none	-	LRAM
.uncached_RAM	* /r_cache* .o (.bss)	Data area without initial value for setting the L1 and L2 caches (see Note2)	-	LRAM
	UNCACHED_BSS	Data area without initial value for buffer.		
.uncached_RAM2	* /r_cache* .o (.text .rodata .data)	Program code area for setting the L1 and L2 caches (see Note2)	FLASH	LRAM
	UNCACHED_DATA	Data area without initial value for buffer.		
.mmu_page_table	none	MMU translation table area	-	LRAM
.stack	none	Stack area for system mode	-	LRAM
		Stack area for IRQ mode		
		Stack area for FIQ mode		
		Stack area for supervisor(SVC) mode		
		Stack area for abort(ABT) mode		
.text2	* (.text .text.*)	Program code area for defaults	FLASH	FLASH
	* (.rodata .rodata.*)	Constant data area for defaults		
.data2	* (.data .data.*)	Data area with initial value for defaults	FLASH	LRAM
.bss2	* (.bss .bss.*)	Data area without initial value for defaults	-	LRAM
	* (COMMON)			
.heap	none	Heap area	-	LRAM
.freertos_heap	none	FreeRTOS heap area	-	LRAM

Notes: 1. "FLASH" and "LRAM" shown in "Loading Area" and "Execution Area" indicate the serial flash memory area and the large-capacity on-chip RAM area respectively.

2. This section should be placed in the cache-disable area.

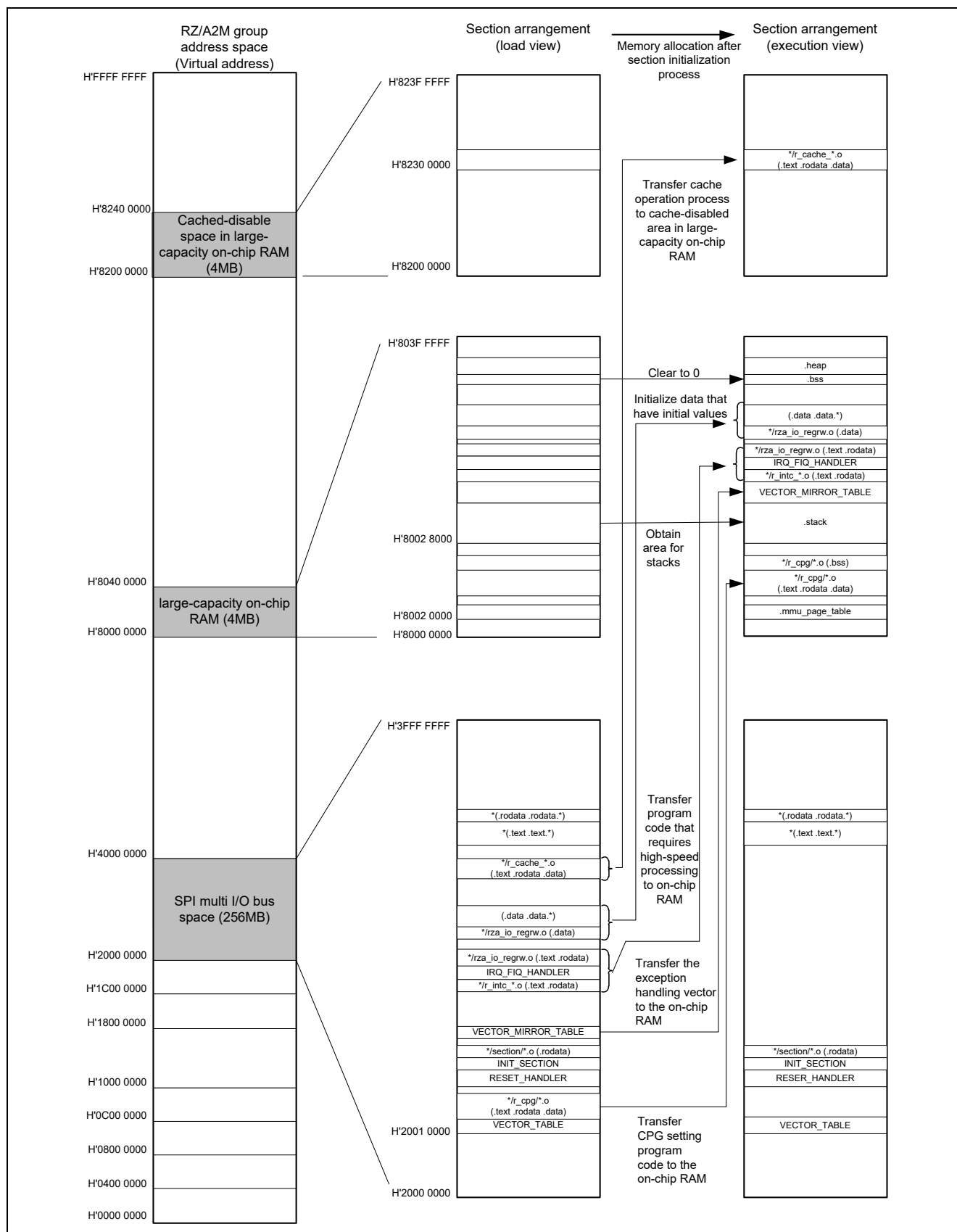


Figure 5.3 Section Assignment

5.3 Interrupts

Table 5.3 shows the interrupts used in the sample program.

Table 5.3 Interrupts used in the sample program

Interrupt Source (Interrupt ID)	Priority	Overview of Interrupt Processing
OSTM0 (88)	31	Fire interrupt every 1ms
RXI4 (322)	30	Fire SCIFA's RXI4 interrupt
TXI4 (323)	30	Fire SCIFA's TXI4 interrupt
INTRIICTEI3 (288)	9	Fire RIIC's INTRIICTEI3 interrupt
INTRIICRI3 (289)	9	Fire RIIC's INTRIICRI3 interrupt
INTRIICTI3 (290)	9	Fire RIIC's INTRIICTI3 interrupt
INTRIICSPI3 (291)	9	Fire RIIC's INTRIICSPI3 interrupt
INTRIICSTI3 (292)	9	Fire RIIC's INTRIICSTI3 interrupt
INTRIICNAKI3 (293)	9	Fire RIIC's INTRIICNAKI3 interrupt
INTRIICALI3 (294)	9	Fire RIIC's INTRIICALI3 interrupt
INTRIICTMOI3 (295)	9	Fire RIIC's INTRIICTMOI3 interrupt

5.4 Fixed-width types

Table 5.4 shows the fixed-width types used in the sample program.

Table 5.4 Fixed-width types used in the sample program

Symbol	Description
char_t	8-bits character
bool_t	Boolean type. Allowable values are true (1) or false (0)
int_t	High-speed signed integer. In the sample code its width is 32 bits
int8_t	Signed 8-bit integer (declared in the standard library stdint.h)
int16_t	Signed 16-bit integer (declared in the standard library stdint.h)
int32_t	Signed 32-bit integer (declared in the standard library stdint.h)
int64_t	Signed 64-bit integer (declared in the standard library stdint.h)
uint8_t	Unsigned 8-bit integer (declared in the standard library stdint.h)
uint16_t	Unsigned 16-bit integer (declared in the standard library stdint.h)
uint32_t	Unsigned 32-bit integer (declared in the standard library stdint.h)
uint64_t	Unsigned 64-bit integer (declared in the standard library stdint.h)
float_t	Single precision floating point number (declared in the standard library math.h)
double_t	Double precision floating point number (declared in the standard library math.h)
float64_t	Double precision floating point number

5.5 Constants

Table 5.5 shows the constants used for EEPROM control of sample program.

Table 5.5 Constants used in the sample code

Constants	Value	Description
NO_ERROR	(0)	EEPROM control is succeeded
ERROR_FAILURE	(-1)	EEPROM control is failed
MAX_SIZE_EEPROM	(16384)	EEPROM (R1EX24128ASAS0A) memory size
EEPROM_DEV_ADDRESS	(0xA0)	EEPROM device slave address
EEPROM_PAGE_SIZE	(64)	EEPROM (R1EX24128ASAS0A) page size
RIIC_CH_EEPROM_RZA2M	(DEVICE_INDENTIFIER "riic3")	RIIC channel name string
EEPROM_DUMMY_READ_BUF_SIZE	(1)	Dummy read data size for ack polling
EEPROM_DUMMY_READ_MAX_TIME	(100)	Max number of repetitions of dummy read for ack polling
MAX_BUF_SIZE_EEPROM_SAMPLE	(256)	Max write size to EEPROM
MAX_FIRST_DATA_EEPROM_SAMPLE	(0xFF)	Max value that can be set as the first data
DATA_COUNT_TYPE_INCREMENT	(0)	Data write mode to EEPROM : increment
DATA_COUNT_TYPE_DECREMENT	(1)	Data write mode to EEPROM : decrement
DATA_COUNT_TYPE_SAME_DATA	(2)	Data write mode to EEPROM : write only the same value
DATA_COUNT_TYPE_MAX	(3)	Upper limit value of write mode setting command
EEPROM_DEFAULT_ADDRESSES	(0x0000)	Default write address to EEPROM
EEPROM_DEFAULT_SIZE	(EEPROM_PAGE_SIZE)	Default write size to EEPROM
EEPROM_DEFAULT_FIRST_DATA	(0x00)	Default value of the first data written to EEPROM
EEPROM_DEFAULT_COUNT_TYPE	(DATA_COUNT_TYPE_INCREMENT)	Default data write mode

Table 5.6 shows the RIIC device driver definition data (const st_r_driver_t) in the sample program. The function whose name begins with "no_dev_" (i.e. no_dev_****) returns with no processing.

Table 5.6 Device Driver Definition Data (const st_r_driver_t)

Constants	Value	Description
g_riic_driver	In the sample program, the following values are implemented: "I2C Device Driver", riic_hld_open, riic_hld_close, no_dev_io, no_dev_write, riic_hld_control, riic_hld_get_version	RIIC driver definition data: Driver name Pointer to the open function Pointer to the close function Pointer to the driver read function Pointer to the driver write function Pointer to the driver control function Pointer to the function to get version info

Table 5.7 shows the constants in RIIC configuration data (static const st_r_drv_riic_sc_config_t), which are used in the sample program. This configuration data is generated by the SC.

Table 5.7 RIIC Configuration Data (static const st_r_drv_riic_sc_config_t)

Constants	Value	Description
RIIC_SC_TABLE[0]	3, { RIIC_MODE_MASTER, true, false, false, 1, 1, 1, RIIC_DEVICE_ADDR_7, RIIC_DEVICE_ADDR_7, RIIC_DEVICE_ADDR_7, RIIC_FREQUENCY_400KHZ, RIIC_DUTY_50, 0, 0, RIIC_FILTER_NOT_USED, RIIC_TIMEOUT_NOT_USED, RIIC_FORAMT_I2C, false, 9, 9, 9, 9, 9, 9, 9, 9, 9 }	Channel number RIIC communication mode RIIC slave address0 enable / disable RIIC slave address0 enable / disable RIIC slave address0 enable / disable RIIC slave address0 value RIIC slave address1 value RIIC slave address2 value RIIC slave address0 address format RIIC slave address1 address format RIIC slave address2 address format SCL frequency SCL duty cycle SCL rise time SCL fall time Noise filter setting Timeout setting I2C Bus format setting Host address detection enable /disable INTRIICTEI3 priority INTRIICRI3 priority INTRIICTI3 priority INTRIICSPI3 priority INTRIICSTI3 priority INTRIICNAKI3 priority INTRIICALI3 priority INTRIICTMOI3 priority

Configuration data (static const `st_r_drv_gpio_sc_config_t`) is generated by SC in the GPIO driver for RIIC control.

Table 5.8 shows the constants in RIIC configuration data (static const `st_r_drv_riic_sc_config_t`), which are used in the sample program. This configuration data is generated by the SC.

Table 5.8 GPIO configuration data for RIIC (const `st_r_drv_gpio_sc_config_t`)

Constants	Value	Description
GPIO_SC_TABLE_riic3	{ {GPIO_PORT_D_PIN_6, {GPIO_FUNC_PERIPHERAL1, GPIO_TINT_DISABLE, GPIO_CURRENT_RESERVE D}, {GPIO_PORT_D_PIN_7, {GPIO_FUNC_PERIPHERAL1, GPIO_TINT_DISABLE, GPIO_CURRENT_RESERVE D}	PD_6 Peripheral function 1 Disable pin interrupts Driving ability is reserved PD_7 Peripheral Function 1 Disable pin interrupts Driving ability is reserved

5.6 Variables

Table 5.9 lists the Constants Used in Sample Code.

Table 5.9 static variables

Type	Variable	Description	Used in
static int_t	gs_handle	Handle for RIIC driver	sample_riic_eeprom_init sample_riic_eeprom_unInit sample_riic_eeprom_write sample_riic_eeprom_read r_riic_ackpoll
static st_cmdfnass_t	gs_cmd_com mand[]	It contains a command to control the sample program The contents are shown below <pre> { "write", cmd_write_eeprom, "address size first_data_value data_count_type<CR> - Write data to EEPROM", }, { "read", cmd_read_eeprom, "address size<CR> - Read data from EEPROM", }, { "help", cmd_help, "<CR> - Show the command description", }, { "ver", cmd_version, "<CR> - Show the application version", }, { "exit", cmd_exit, "<CR> - Exit sample program", } </pre>	cmd_console

5.7 Functions

For EEPROM control, the following functions are implemented in the sample program:

- EEPROM Control Sample Function for writing/reading EEPROM
- Console Function accepting operation command as user interface

In the sample code, the IO register-write and register-read function is used when accessing peripheral IO registers in bit unit.

Table 5.10 lists EEPROM Control Sample Function, and Table 5.11 lists Console Function, respectively.

Table 5.10 EEPROM Control Sample Function

Function Name	Description
sample_riic_eeprom_init	Sample processing of open RIIC driver for EEPROM control
sample_riic_eeprom_unInit	Sample processing of close RIIC driver
sample_riic_eeprom_write	Sample processing of writing data to EEPROM
sample_riic_eeprom_read	Sample processing of reading data from EEPROM
r_riic_ackpoll	Wait for ACK polling from EEPROM
init_eeprom	Initialize the RIIC driver for EEPROM control

Table 5.11 Console Function

Function Name	Description
os_console_task_t	Task function to call console function
cmd_console	It accepts the control command and calls each operation function
cmd_version	Show version information
cmd_write_eeprom	Execute write operation command to EEPROM
cmd_read_eeprom	Execute read operation command from EEPROM
cmd_help	Show descriptions of each operation command
cmd_exit	Quit the console

5.8 Function Specification

This section describes the specification of function in the sample code for EEPROM control.

5.8.1 EEPROM Control Function

Specifications of EEPROM Control Function is shown below.

sample_riic_eeprom_init	
Overview	Initialize processing of sample program
Syntax	int_t sample_riic_eeprom_init (void)
Description	Initialize RIIC driver for EEPROM control This function calls open() to initialize the RIIC driver and peripherals. Peripherals of RIIC are set to the state shown in Table 5.1
Parameters	None
Return value	NO_ERROR: Initialize is succeeded ERROR_FAILURE: Initialize is failed
sample_riic_eeprom_uninit	
Overview	Uninitialize processing of sample program
Syntax	int_t sample_riic_eeprom_uninit (void)
Description	Uninitialize RIIC driver This function calls close() to uninitialize the RIIC driver.
Parameters	None
Return value	NO_ERROR: always return
sample_riic_eeprom_write	
Overview	Write processing of sample program
Syntax	int_t sample_riic_eeprom_write(uint16_t addr, const void *data, uint32_t size)
Description	Perform write processing to EEPROM This function calls control() to write data
Parameters	addr EEPROM write address *data Pointer to write data buffer size Write data size
Return value	NO_ERROR: Data write process is succeeded ERROR_FAILURE: Data write process is failed
sample_riic_eeprom_read	
Overview	Read processing of sample program
Syntax	int_t sample_riic_eeprom_read(uint16_t addr, const void *data, uint32_t size)
Description	Perform read processing from EEPROM This function calls control() to read data
Parameters	addr EEPROM read address *data Pointer to read data buffer size Read data size
Return value	NO_ERROR: Data read process is succeeded ERROR_FAILURE: Data read process is failed

r_riic_ackpoll	
Overview	Wait processing of writing completion to EEPROM
Syntax	int_t r_riic_ackpoll(void)
Description	Confirm the end of writing by repeating dummy read from EEPROM until it succeeds
Parameters	None
Return value	NO_ERROR: ACK polling complete ERROR_FAILURE: ACK polling timeout

init_eeprom	
Overview	Initialize the EEPROM on the main task
Syntax	int_t init_eeprom(void)
Description	Call sample_riic_eeprom_init on the main task to initialize the EEPROM control process
Parameters	None
Return value	NO_ERROR: Initialize is succeeded ERROR_FAILURE: Initialize is failed

5.8.2 Console Function

Specifications of Console Function is shown below.

os_console_task_t	
Overview	Call console function
Syntax	int_t os_console_task_t(void)
Description	Show usable console command. After that, call cmd_console() to start accepting commands.
Parameters	None
Return value	No return (infinite loop)

cmd_console	
Overview	Command acceptance process
Syntax	void cmd_console(FILE* pIn, FILE *pOut, char *pszPrompt)
Description	It waits for a command and calls a function corresponding to the received command.
Parameters	pIn Pointer to the input file stream pOut Pointer to the file stream for echo of command input *pszPrompt Pointer to a command prompt
Return value	None

cmd_version	
Overview	Show version information
Syntax	int16_t cmd_version(int_t iArgCount, char_t **ppszArgument, pst_comset_t pCom)
Description	This function shows the version information of the sample program to the terminal software.
Parameters	iArgCount The number of arguments in the argument list **ppszArgument The Argument list pCom Pointer to the command object
Return value	CMD_OK: always return

cmd_write_eeprom							
Overview	Execute operation of writing to EEPROM						
Syntax	int16_t cmd_write_eeprom(int_t iArgCount, char_t **ppszArgument, pst_comset_t pCom)						
Description	This function receives write command and argument such as write address and check if the argument is correct. After checking arguments, call sample_riic_eeprom_write() to execute data write to EEPROM. The expected command is “write ‘address’ ‘size’ ‘data’ ‘type’”, and if arguments are missing, write processing is performed by setting default in arguments. Range of each argument: (address + size): 0x0000 - MAX_SIZE_EEPROM size: MAX_BUF_SIZE_EEPROM_SAMPLE data: 0x00 - MAX_FIRST_DATA_EEPROM_SAMPLE type: DATA_COUNT_TYPE_INCREMENT, DATA_COUNT_TYPE_DECREMENT, DATA_COUNT_TYPE_SAMEDATA						
Parameters	<table> <tr> <td>iArgCount</td><td>The number of arguments in the argument list</td></tr> <tr> <td>**ppszArgument</td><td>The Argument list</td></tr> <tr> <td>pCom</td><td>Pointer to the command object</td></tr> </table>	iArgCount	The number of arguments in the argument list	**ppszArgument	The Argument list	pCom	Pointer to the command object
iArgCount	The number of arguments in the argument list						
**ppszArgument	The Argument list						
pCom	Pointer to the command object						
Return value	CMD_OK: always return						
cmd_read_eeprom							
Overview	Execute operation of reading from EEPROM						
Syntax	int16_t cmd_read_eeprom(int_t iArgCount, char_t **ppszArgument, pst_comset_t pCom)						
Description	This function receives read command and argument such as read address and check if the argument is correct. After checking arguments, call sample_riic_eeprom_read() to execute data read from EEPROM. The expected command is “write ‘address’ ‘size’”, and if arguments are missing, read processing is performed by setting default in arguments. Range of each argument: (address + size): 0x0000 - MAX_SIZE_EEPROM size: MAX_BUF_SIZE_EEPROM_SAMPLE						
Parameters	<table> <tr> <td>iArgCount</td><td>The number of arguments in the argument list</td></tr> <tr> <td>**ppszArgument</td><td>The Argument list</td></tr> <tr> <td>pCom</td><td>Pointer to the command object</td></tr> </table>	iArgCount	The number of arguments in the argument list	**ppszArgument	The Argument list	pCom	Pointer to the command object
iArgCount	The number of arguments in the argument list						
**ppszArgument	The Argument list						
pCom	Pointer to the command object						
Return value	CMD_OK: always return						
cmd_help							
Overview	Show command help						
Syntax	int16_t cmd_help (int_t iArgCount, char_t **ppszArgument, pst_comset_t pCom)						
Description	This function shows the command, the argument it requires, the process to be executed.						
Parameters	<table> <tr> <td>iArgCount</td><td>The number of arguments in the argument list</td></tr> <tr> <td>**ppszArgument</td><td>The Argument list</td></tr> <tr> <td>pCom</td><td>Pointer to the command object</td></tr> </table>	iArgCount	The number of arguments in the argument list	**ppszArgument	The Argument list	pCom	Pointer to the command object
iArgCount	The number of arguments in the argument list						
**ppszArgument	The Argument list						
pCom	Pointer to the command object						
Return value	CMD_OK: always return						

cmd_exit	
Overview	Exit process the console
Syntax	int16_t cmd_exit (int_t iArgCount, char_t **ppszArgument, pst_comset_t pCom)
Description	Return the return value(CMD_LOG_OUT) for terminating the console to cmd_console.
Parameters	iArgCount The number of arguments in the argument list
	**ppszArgument The Argument list
	pCom Pointer to the command object
Return value	CMD_LOG_OUT: always return

5.9 Flowcharts

5.9.1 sample_riic_eeprom_init

Figure 5.4 shows the flowchart of sample_riic_eeprom_init.

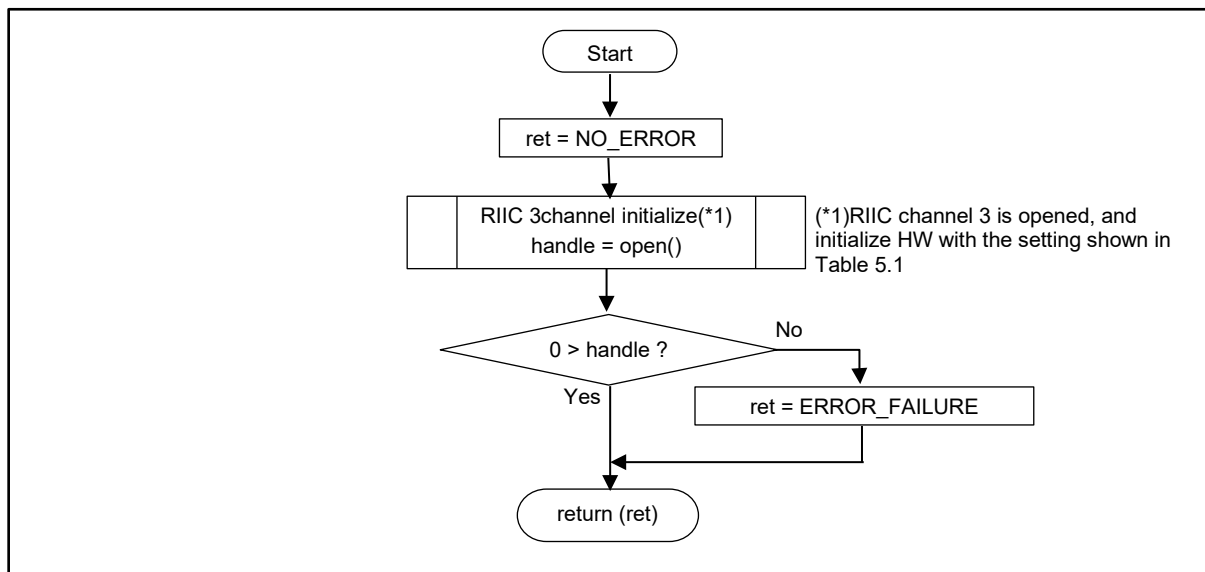


Figure 5.4 sample_riic_eeprom_init

5.9.2 sample_riic_eeprom_uninit

Figure 5.5 shows the flowchart of sample_riic_eeprom_uninit.

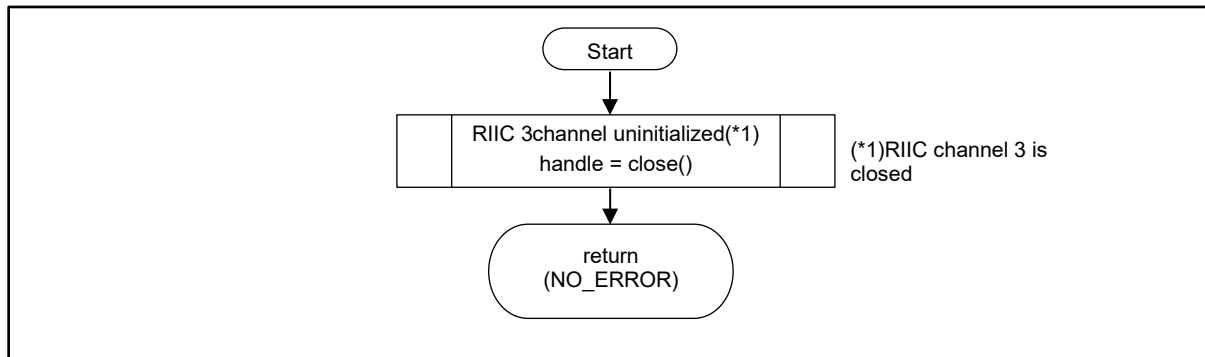


Figure 5.5 sample_riic_eeprom_uninit

5.9.3 sample_riic_eeprom_write

Figure 5.6 shows the flowchart of sample_riic_eeprom_write.

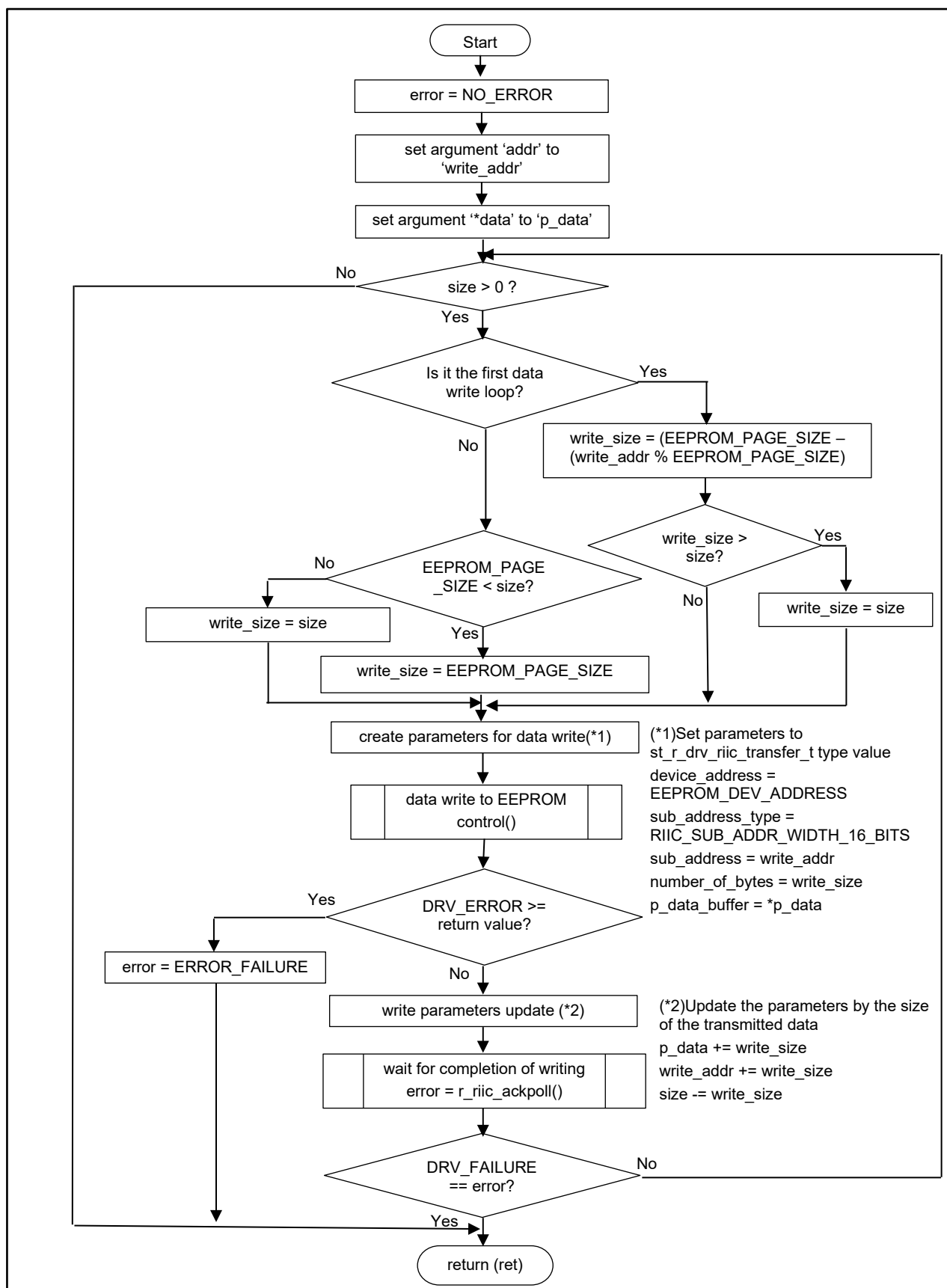


Figure 5.6 sample_riic_eeprom_write

5.9.4 sample_riic_eeprom_read

Figure 5.7 shows the flowchart of sample_riic_eeprom_read.

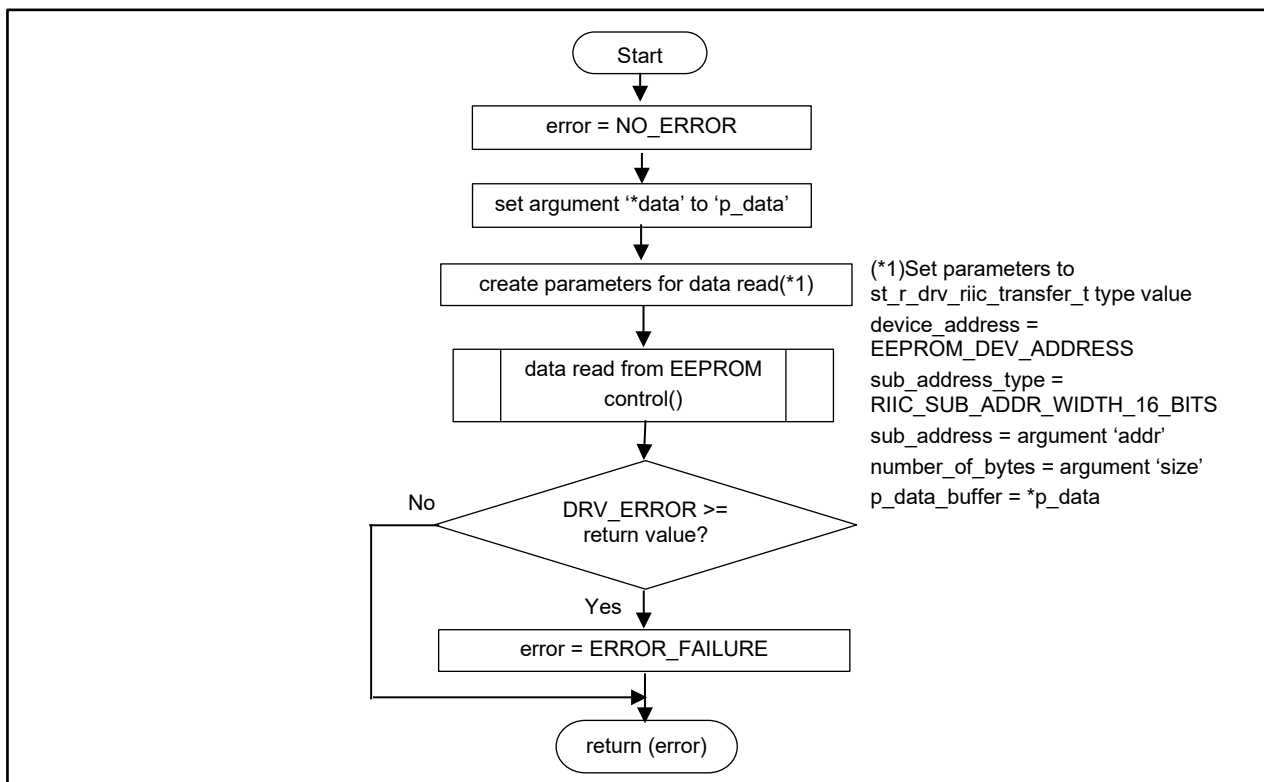


Figure 5.7 sample_riic_eeprom_read

5.9.5 r_riic_ackpoll

Figure 5.8 shows the flowchart of r_riic_ackpoll

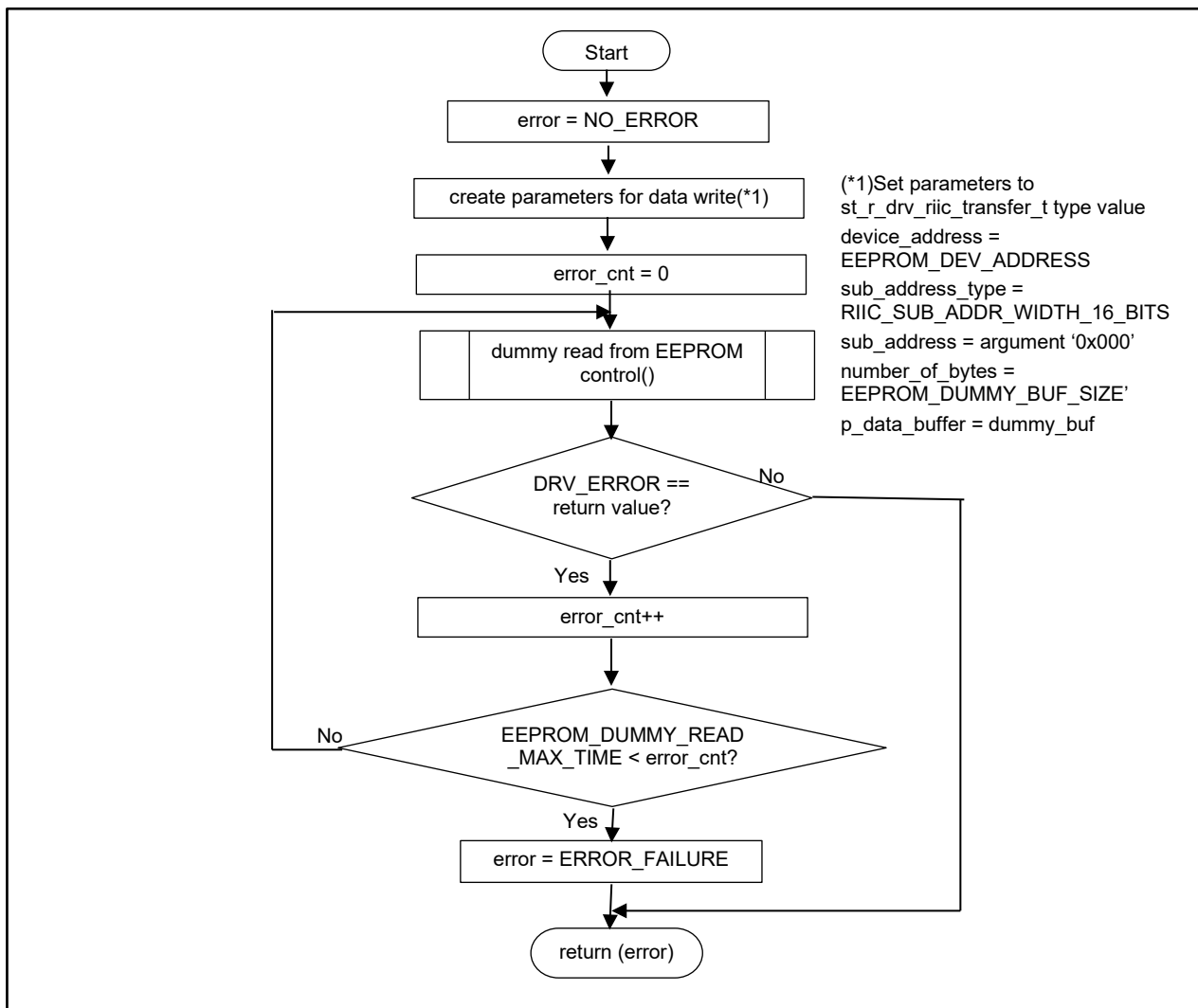


Figure 5.8 r_riic_ackpoll

5.9.6 cmd_version

Figure 5.9 shows the flowchart of cmd_version

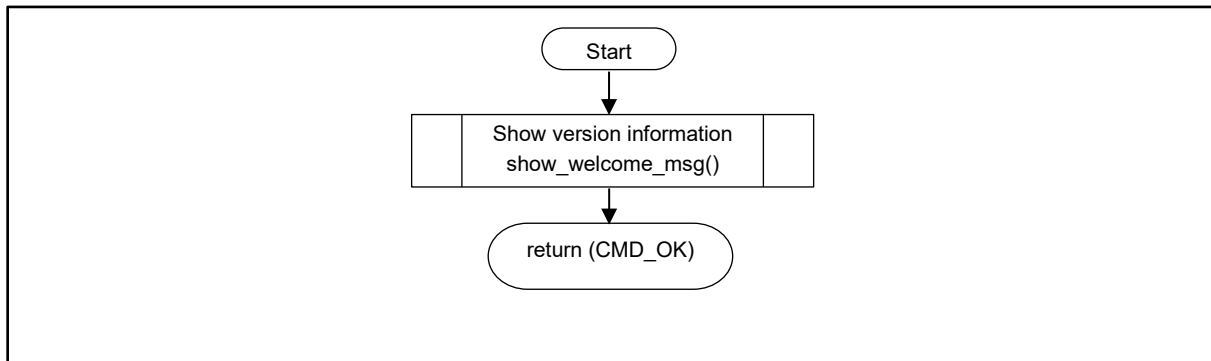


Figure 5.9 cmd_version

5.9.7 cmd_write_eeprom

Figure 5.10 shows the flowchart of cmd_write_eeprom.

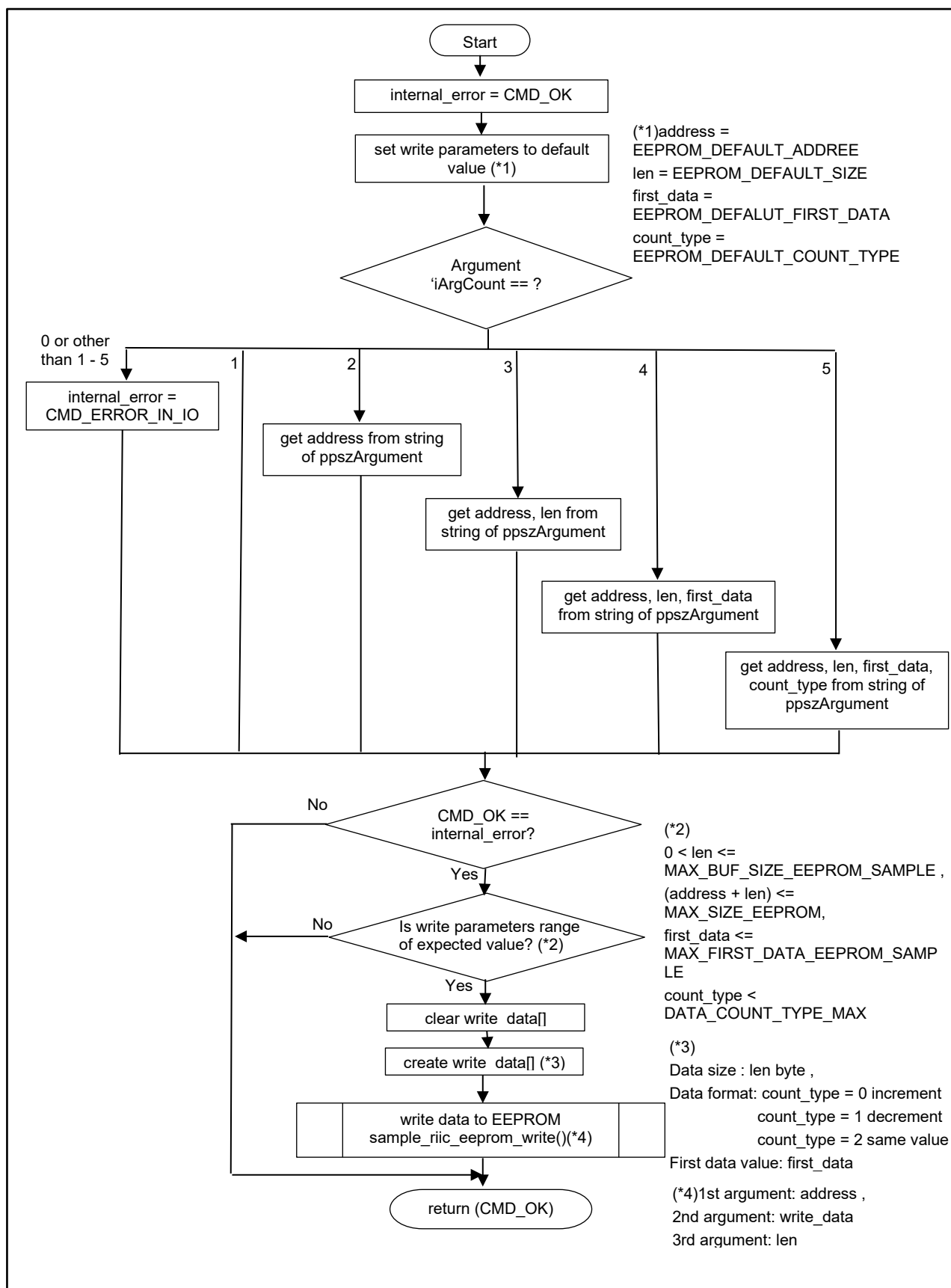


Figure 5.10 cmd_write_eeprom

5.9.8 cmd_read_eeprom

Figure 5.11 shows the flowchart of cmd_read_eeprom.

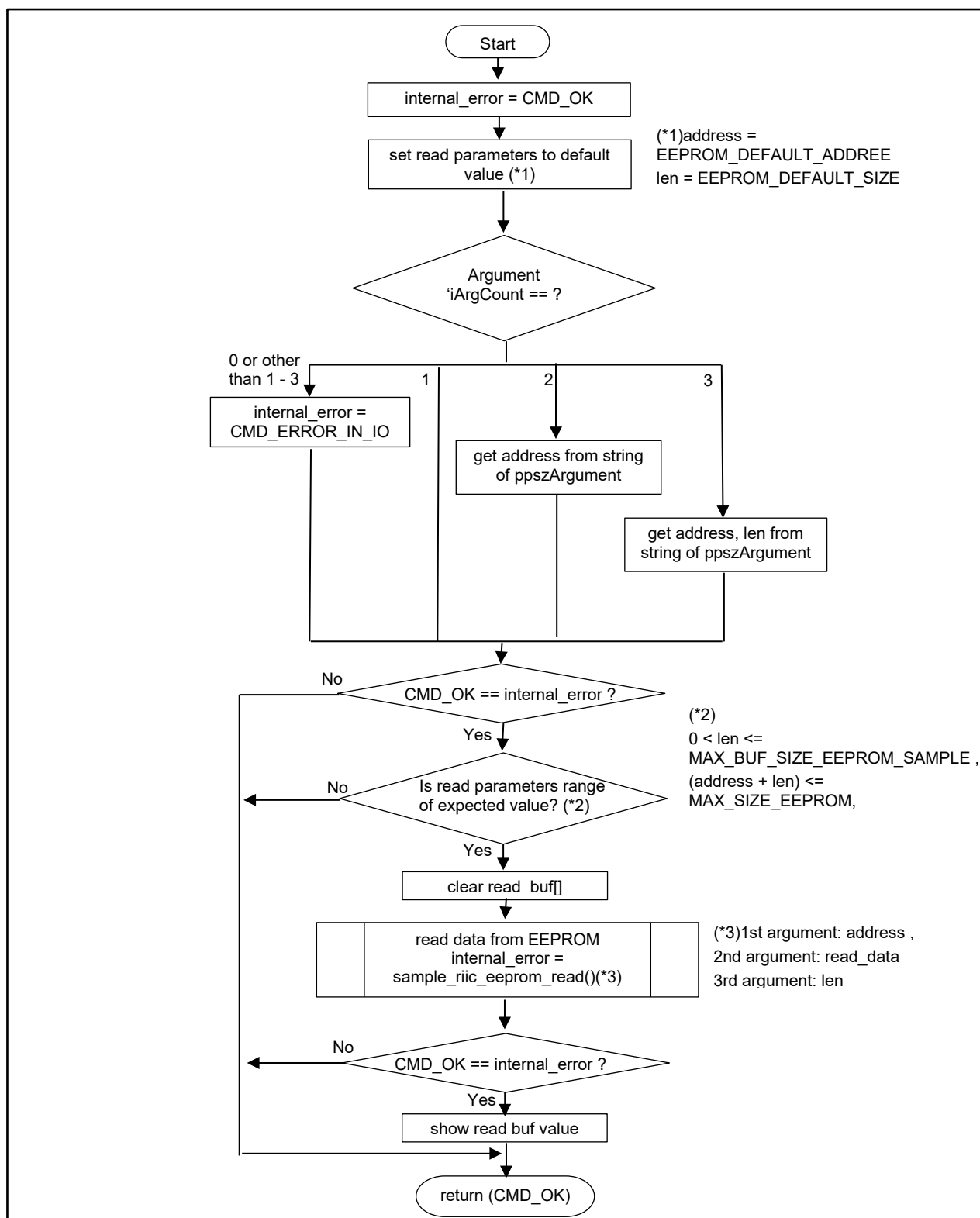


Figure 5.11 cmd_read_eeprom

5.9.9 cmd_help

Figure 5.12 shows the flowchart of cmd_version

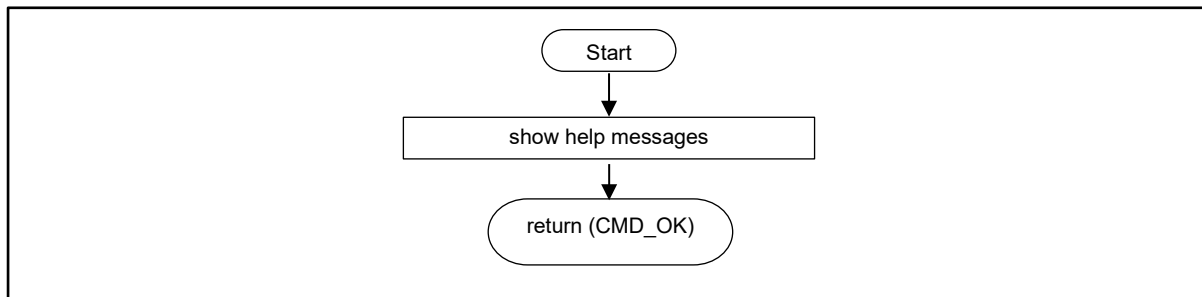


Figure 5.12 cmd_help

5.9.10 cmd_exit

Figure 5.13 shows the flowchart of cmd_exit.

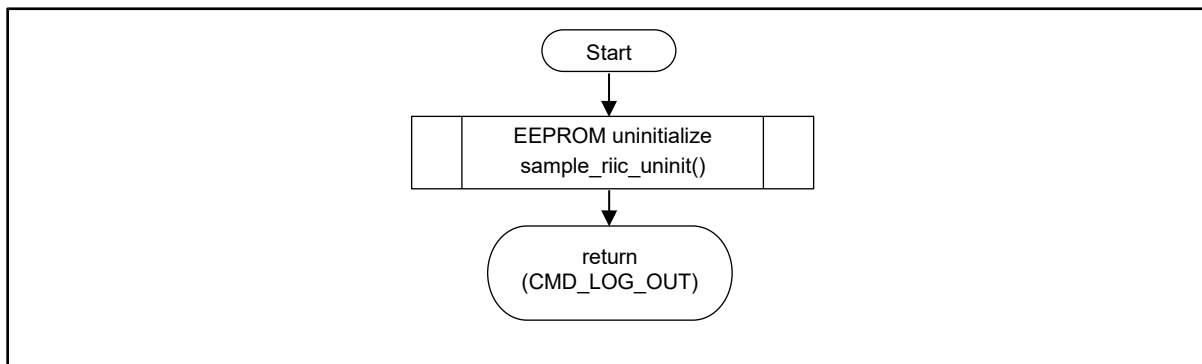


Figure 5.13 cmd_exit

6. Sample Code

Sample code can be downloaded from the Renesas Electronics website.

7. Reference Documents

User's Manual: Hardware

RZ/A2M Group User's Manual: Hardware

The latest version can be downloaded from the Renesas Electronics website.

RTX921053C00000BE (RZ/A2M CPU board) User's Manual

The latest version can be downloaded from the Renesas Electronics website.

RTK79210XXB000000BE (RZ/A2M SUB board) User's Manual

The latest version can be downloaded from the Renesas Electronics website.

R1EX24128ASAS0A Two-wire serial interface 128k EEPROM (16-kword * 8-bit) Datasheet

The latest version can be downloaded from the ARM website.

ARM Architecture Reference Manual ARMv7-A and ARMv7-R edition Issue C

The latest version can be downloaded from the ARM website.

ARM Cortex™-A9 Technical Reference Manual Revision: r4p1

The latest version can be downloaded from the ARM website.

ARM Generic Interrupt Controller Architecture Specification - Architecture version 2.0

The latest version can be downloaded from the ARM website.

ARM CoreLink™ Level 2 Cache Controller L2C-310 Technical Reference Manual Revision: r3p3

The latest version can be downloaded from the ARM website.

Technical Update/Technical News

The latest information can be downloaded from the Renesas Electronics website.

User's Manual: Development Tools

Integrated development environment e² studio User's Manual can be downloaded from the Renesas Electronics website.

The latest version can be downloaded from the Renesas Electronics website.

Revision History

Rev.	Date	Description	
		Page	Summary
Rev.1.00	Dec 28, 2018	—	First edition issued
Rev.1.01	Apr 15, 2019	7	Table 1.1 Modified I2C communication target mistake “RZ/A2M SUB board”->“EEPROM”
		9	Table 2.1 Version of e2studio for operation is updated
		15, 19	Table 5.1, Table 5.3 Modified the information of the used interrupt
		23	Table 5.9 Modified the variable name
		23 to 36	Change the function name of EEPROM control functions
Rev.1.10	May 17, 2019	8	Table 2.1 Remove compiler option "-mthumb-interwork"
Rev.1.11	Sep. 30, 2020	8	Table 2.1 Remove compiler option about warnings.
Rev.1.12	Jan. 20, 2022	8	Update the version of complier. Update the version of e2 studio.
Rev.1.13	Apr. 22, 2024	8	Update the version of complier. Update the version of e2 studio.
Rev.1.14	Jun. 30, 2025		Update interrupt handlers to call portYIELD_FROM_ISR after calling FreeRTOS API ended with fromISR.
Rev.1.15	Mar. 13, 2026	8	Update the version of e2 studio.
		-	Enable breakpoint settings for some assembly (asm) files. Update the write operation of IRQRR register to prevent incorrect IRQ interrupt request register clearing.

General Precautions in the Handling of Microprocessing Unit and Microcontroller Unit Products

The following usage notes are applicable to all Microprocessing unit and Microcontroller unit products from Renesas. For detailed usage notes on the products covered by this document, refer to the relevant sections of the document as well as any technical updates that have been issued for the products.

1. Precaution against Electrostatic Discharge (ESD)

A strong electrical field, when exposed to a CMOS device, can cause destruction of the gate oxide and ultimately degrade the device operation. Steps must be taken to stop the generation of static electricity as much as possible, and quickly dissipate it when it occurs. Environmental control must be adequate. When it is dry, a humidifier should be used. This is recommended to avoid using insulators that can easily build up static electricity.

Semiconductor devices must be stored and transported in an anti-static container, static shielding bag or conductive material. All test and measurement tools including work benches and floors must be grounded. The operator must also be grounded using a wrist strap. Semiconductor devices must not be touched with bare hands. Similar precautions must be taken for printed circuit boards with mounted semiconductor devices.

2. Processing at power-on

The state of the product is undefined at the time when power is supplied. The states of internal circuits in the LSI are indeterminate and the states of register settings and pins are undefined at the time when power is supplied. In a finished product where the reset signal is applied to the external reset pin, the states of pins are not guaranteed from the time when power is supplied until the reset process is completed. In a similar way, the states of pins in a product that is reset by an on-chip power-on reset function are not guaranteed from the time when power is supplied until the power reaches the level at which resetting is specified.

3. Input of signal during power-off state

Do not input signals or an I/O pull-up power supply while the device is powered off. The current injection that results from input of such a signal or I/O pull-up power supply may cause malfunction and the abnormal current that passes in the device at this time may cause degradation of internal elements. Follow the guideline for input signal during power-off state as described in your product documentation.

4. Handling of unused pins

Handle unused pins in accordance with the directions given under handling of unused pins in the manual. The input pins of CMOS products are generally in the high-impedance state. In operation with an unused pin in the open-circuit state, extra electromagnetic noise is induced in the vicinity of the LSI, an associated shoot-through current flows internally, and malfunctions occur due to the false recognition of the pin state as an input signal become possible.

5. Clock signals

After applying a reset, only release the reset line after the operating clock signal becomes stable. When switching the clock signal during program execution, wait until the target clock signal is stabilized. When the clock signal is generated with an external resonator or from an external oscillator during a reset, ensure that the reset line is only released after full stabilization of the clock signal. Additionally, when switching to a clock signal produced with an external resonator or by an external oscillator while program execution is in progress, wait until the target clock signal is stable.

6. Voltage application waveform at input pin

Waveform distortion due to input noise or a reflected wave may cause malfunction. If the input of the CMOS device stays in the area between V_{IL} (Max.) and V_{IH} (Min.) due to noise, for example, the device may malfunction. Take care to prevent chattering noise from entering the device when the input level is fixed, and also in the transition period when the input level passes through the area between V_{IL} (Max.) and V_{IH} (Min.).

7. Prohibition of access to reserved addresses

Access to reserved addresses is prohibited. The reserved addresses are provided for possible future expansion of functions. Do not access these addresses as the correct operation of the LSI is not guaranteed.

8. Differences between products

Before changing from one product to another, for example to a product with a different part number, confirm that the change will not lead to problems. The characteristics of a microprocessing unit or microcontroller unit products in the same group but having a different part number might differ in terms of internal memory capacity, layout pattern, and other factors, which can affect the ranges of electrical characteristics, such as characteristic values, operating margins, immunity to noise, and amount of radiated noise. When changing to a product with a different part number, implement a system-evaluation test for the given product.

Notice

1. Descriptions of circuits, software and other related information in this document are provided only to illustrate the operation of semiconductor products and application examples. You are fully responsible for the incorporation or any other use of the circuits, software, and information in the design of your product or system. Renesas Electronics disclaims any and all liability for any losses and damages incurred by you or third parties arising from the use of these circuits, software, or information.
2. Renesas Electronics hereby expressly disclaims any warranties against and liability for infringement or any other claims involving patents, copyrights, or other intellectual property rights of third parties, by or arising from the use of Renesas Electronics products or technical information described in this document, including but not limited to, the product data, drawings, charts, programs, algorithms, and application examples.
3. No license, express, implied or otherwise, is granted hereby under any patents, copyrights or other intellectual property rights of Renesas Electronics or others.
4. You shall be responsible for determining what licenses are required from any third parties, and obtaining such licenses for the lawful import, export, manufacture, sales, utilization, distribution or other disposal of any products incorporating Renesas Electronics products, if required.
5. You shall not alter, modify, copy, or reverse engineer any Renesas Electronics product, whether in whole or in part. Renesas Electronics disclaims any and all liability for any losses or damages incurred by you or third parties arising from such alteration, modification, copying or reverse engineering.
6. Renesas Electronics products are classified according to the following two quality grades: "Standard" and "High Quality". The intended applications for each Renesas Electronics product depends on the product's quality grade, as indicated below.

"Standard": Computers; office equipment; communications equipment; test and measurement equipment; audio and visual equipment; home electronic appliances; machine tools; personal electronic equipment; industrial robots; etc.

"High Quality": Transportation equipment (automobiles, trains, ships, etc.); traffic control (traffic lights); large-scale communication equipment; key financial terminal systems; safety control equipment; etc.

Unless expressly designated as a high reliability product or a product for harsh environments in a Renesas Electronics data sheet or other Renesas Electronics document, Renesas Electronics products are not intended or authorized for use in products or systems that may pose a direct threat to human life or bodily injury (artificial life support devices or systems; surgical implantations; etc.), or may cause serious property damage (space system; undersea repeaters; nuclear power control systems; aircraft control systems; key plant systems; military equipment; etc.). Renesas Electronics disclaims any and all liability for any damages or losses incurred by you or any third parties arising from the use of any Renesas Electronics product that is inconsistent with any Renesas Electronics data sheet, user's manual or other Renesas Electronics document.

7. No semiconductor product is absolutely secure. Notwithstanding any security measures or features that may be implemented in Renesas Electronics hardware or software products, Renesas Electronics shall have absolutely no liability arising out of any vulnerability or security breach, including but not limited to any unauthorized access to or use of a Renesas Electronics product or a system that uses a Renesas Electronics product. RENESAS ELECTRONICS DOES NOT WARRANT OR GUARANTEE THAT RENESAS ELECTRONICS PRODUCTS, OR ANY SYSTEMS CREATED USING RENESAS ELECTRONICS PRODUCTS WILL BE INVULNERABLE OR FREE FROM CORRUPTION, ATTACK, VIRUSES, INTERFERENCE, HACKING, DATA LOSS OR THEFT, OR OTHER SECURITY INTRUSION ("Vulnerability Issues"). RENESAS ELECTRONICS DISCLAIMS ANY AND ALL RESPONSIBILITY OR LIABILITY ARISING FROM OR RELATED TO ANY VULNERABILITY ISSUES. FURTHERMORE, TO THE EXTENT PERMITTED BY APPLICABLE LAW, RENESAS ELECTRONICS DISCLAIMS ANY AND ALL WARRANTIES, EXPRESS OR IMPLIED, WITH RESPECT TO THIS DOCUMENT AND ANY RELATED OR ACCOMPANYING SOFTWARE OR HARDWARE, INCLUDING BUT NOT LIMITED TO THE IMPLIED WARRANTIES OF MERCHANTABILITY, OR FITNESS FOR A PARTICULAR PURPOSE.
8. When using Renesas Electronics products, refer to the latest product information (data sheets, user's manuals, application notes, "General Notes for Handling and Using Semiconductor Devices" in the reliability handbook, etc.), and ensure that usage conditions are within the ranges specified by Renesas Electronics with respect to maximum ratings, operating power supply voltage range, heat dissipation characteristics, installation, etc. Renesas Electronics disclaims any and all liability for any malfunctions, failure or accident arising out of the use of Renesas Electronics products outside of such specified ranges.
9. Although Renesas Electronics endeavors to improve the quality and reliability of Renesas Electronics products, semiconductor products have specific characteristics, such as the occurrence of failure at a certain rate and malfunctions under certain use conditions. Unless designated as a high reliability product or a product for harsh environments in a Renesas Electronics data sheet or other Renesas Electronics document, Renesas Electronics products are not subject to radiation resistance design. You are responsible for implementing safety measures to guard against the possibility of bodily injury, injury or damage caused by fire, and/or danger to the public in the event of a failure or malfunction of Renesas Electronics products, such as safety design for hardware and software, including but not limited to redundancy, fire control and malfunction prevention, appropriate treatment for aging degradation or any other appropriate measures. Because the evaluation of microcomputer software alone is very difficult and impractical, you are responsible for evaluating the safety of the final products or systems manufactured by you.
10. Please contact a Renesas Electronics sales office for details as to environmental matters such as the environmental compatibility of each Renesas Electronics product. You are responsible for carefully and sufficiently investigating applicable laws and regulations that regulate the inclusion or use of controlled substances, including without limitation, the EU RoHS Directive, and using Renesas Electronics products in compliance with all these applicable laws and regulations. Renesas Electronics disclaims any and all liability for damages or losses occurring as a result of your noncompliance with applicable laws and regulations.
11. Renesas Electronics products and technologies shall not be used for or incorporated into any products or systems whose manufacture, use, or sale is prohibited under any applicable domestic or foreign laws or regulations. You shall comply with any applicable export control laws and regulations promulgated and administered by the governments of any countries asserting jurisdiction over the parties or transactions.
12. It is the responsibility of the buyer or distributor of Renesas Electronics products, or any other party who distributes, disposes of, or otherwise sells or transfers the product to a third party, to notify such third party in advance of the contents and conditions set forth in this document.
13. This document shall not be reprinted, reproduced or duplicated in any form, in whole or in part, without prior written consent of Renesas Electronics.
14. Please contact a Renesas Electronics sales office if you have any questions regarding the information contained in this document or Renesas Electronics products.

(Note1) "Renesas Electronics" as used in this document means Renesas Electronics Corporation and also includes its directly or indirectly controlled subsidiaries.

(Note2) "Renesas Electronics product(s)" means any product developed or manufactured by or for Renesas Electronics.

(Rev.5.0-1 October 2020)

Corporate Headquarters

TOYOSU FORESIA, 3-2-24 Toyosu,
Koto-ku, Tokyo 135-0061, Japan
www.renesas.com

Trademarks

Renesas and the Renesas logo are trademarks of Renesas Electronics Corporation. All trademarks and registered trademarks are the property of their respective owners.

Contact information

For further information on a product, technology, the most up-to-date version of a document, or your nearest sales office, please visit:

www.renesas.com/contact/.